

Conceptual Pipeline Design - Beejan Technologies

1. **Source Identification:** The pipeline will collect complaints from the four main sources: social media, SMS, call log files, and website forms. Like JSON from social media APIs, text files from call logs and SMS, and CSV files from website submissions. Since complaints arrive continuously, the pipeline must be designed to handle both batch and near real-time streams (micro batch).
2. **Ingestion Strategy:** Ingestion will depend on the source type. Social media data will be pulled in through APIs, using micro-batching to capture complaints at frequent intervals. Call logs, SMS, and website complaints will be ingested in batch mode through scheduled file uploads, ensuring the data is captured consistently and ready for processing at regular intervals.
3. **Processing/Transformation:** The raw data will first go in a data lake. Cleaning and standardization steps will then run to ensure consistency; all dates will be converted to a single format, missing values filled or flagged, duplicates removed, and categorical fields normalized. Complaint categories such as 'poor network', 'incorrect billing', or 'bad customer service' will be assigned using keyword matching, assuming the complaint text will be primarily in English and basic keyword matching will achieve 80-85% accuracy for categorization. Numeric fields will be standardized as integers or floats.
4. **Storage Options:** Raw data will stay in the data lake, in its original form. After being processed and cleaned, the data will be loaded into a data warehouse since it supports structured data mainly and that is best for reporting and analysis. Parquet will be used as the main storage format in the warehouse because it compresses well and supports fast queries on large datasets.
5. **Serving:** The data warehouse will act as the main serving layer. The reporting team will query it directly to create dashboards and reports. Data analysts and data scientists will work with it using query languages or programming environments for deeper analysis. Software teams will access it through APIs to build applications that depend on complaint data.
6. **Orchestration & Monitoring:** The pipeline will run every 4 hours, totaling six runs each

day. Using a cloud-based orchestration tool will schedule the runs, track each task, and manage all dependencies. Monitoring will track every job with checkpoints that log whether it succeeds or fails. If a job fails because of a temporary issue, it will retry automatically. Alerts will be sent via email if any stage fails, ensuring issues are caught quickly.

7. **DataOps:** The pipeline is first built and tested to ensure it functions as expected. Once validated, it is deployed to a cloud production environment to process real complaint data. To maintain reliability and efficiency, DataOps practices are applied, like version control for code, automated tests to validate data quality, and access controls to protect sensitive information. The pipeline is continuously refined and improved over time based on feedback from downstream users.

Challenges and Unknowns

- We don't know the exact APIs, file formats, or data schemas from each source system, which affects the detailed ingestion design.
- Critical issues like network outages might need faster alerts, but we don't know the exact urgency requirements for different complaint types.
- Uncertain about actual processing times and storage growth, making it difficult to estimate infrastructure needs and costs.
- All downstream users may have varying data access needs and reporting frequencies that aren't fully defined yet.
- Since there are unknown complaint volumes, peak patterns, and customer ID consistency across channels that could impact pipeline performance, I assumed typical business patterns with 2-3 day resolution cycles, and consistent customer identification for effective linking.